Proceedings of Machine Learning Research vol vvv:1–25, 2027

Atlas-DeePC: Learning Behavioral Atlases for Predictive Control

Dian Jin, Zhiyuan Jin, Jeremy Coulson

Abstract

Data-enabled Predictive Control (DeePC) uses linear combinations of recorded input-output trajectories to compute control actions without a parametric model. For nonlinear systems, these linear combinations may not satisfy the system dynamics, and the computational cost grows with dataset size. We propose Atlas-DeePC, which approximates the behavior of a dynamical system using a collection of low-dimensional subspaces, termed a behavioral atlas, to enable computationally efficient prediction and control. The atlas is constructed offline by clustering the data and fitting a subspace to each cluster. Online, the controller selects the subspace that best matches recent measurements and solves a predictive control problem parameterized by its orthonormal basis. A graph-based reference planner complements the controller by providing intermediate references across operating regions. We illustrate Atlas-DeePC on a planar robotic manipulator, where it achieves higher success rates and substantially lower online computation times than the compared controllers based on online data selection. An additional benchmark of an autonomous racing task is also included. The code is publicly available[1].

Keywords: Data-driven control; nonlinear systems; robotic control

1. Introduction

Controlling nonlinear systems requires handling dynamics that change across operating regions while respecting constraints. Model predictive control addresses constraints explicitly (Camacho and Bordons, 2007), but it relies on an accurate model, which can be difficult to derive. Data-driven control methods instead design controllers directly from input-output data, bypassing explicit model identification. Among these approaches, Data-enabled Predictive Control (DeePC) represents the trajectories of a Linear Time-Invariant (LTI) system as a linear subspace spanned by sufficiently informative input-output data (Willems et al., 2005), and uses linear combinations of these trajectories to predict future trajectories in a receding-horizon controller (Coulson et al., 2019; Berberich et al., 2021). However, the input-output trajectories of a nonlinear system generally do not form a linear subspace, and the computational cost grows with the size of the dataset.

Existing extensions of data-driven predictive control to nonlinear systems follow two broad strategies (Berberich and Allgöwer, 2025): exploiting linearity in higher-dimensional coordinates, as in kernel and Koopman formulations (Korda and Mezić, 2018; Huang et al., 2024), or exploiting local linearity of the dynamics, as in predictors built from selected data near the current operating point (Berberich et al., 2022a; Näf et al., 2025; Beerwerth and Alrifaee, 2026). Global representations require choosing kernels or observables rich enough to capture the nonlinearity across all operating regions, which is difficult without prior knowledge of the system (Brunton et al., 2016). Local approaches select the data most relevant to the current operating point (Näf et al., 2025; Beerwerth and Alrifaee, 2026), which requires a search over the full dataset online. Alternatively, the data can be partitioned offline into local datasets, either according to a prescribed scheduling variable (Guerrero et al., 2025; Zieglmeier et al., 2025) or by clustering input-output trajectories into

1. https://github.com/DianJin-Frederick/atlas-deepc

© 2027 D. Jin, Z. Jin & J. Coulson.

local linear sub-behaviors, as in big data-driven predictive control (BDPC) (Han et al., 2024). The former requires system knowledge to select a scheduling variable that captures the dominant nonlinearity (Zieglmeier et al., 2025, Assumption 2). BDPC approaches construct local sub-behaviors for one-step predictive control (Han et al., 2024), with extensions that impose contraction conditions (Han et al., 2025) or interpolate sub-behaviors and select paths based on uncertainty (Han et al., 2026). Their one-step formulation (Han et al., 2026, Eq. (34)) limits the controller's prediction horizon and may impair tracking performance.

We develop Atlas-DeePC, which represents the data by a collection of local low-dimensional subspaces, termed *behavioral charts*, which together form a *behavioral atlas* by analogy with manifold atlases (Lee, 2012). Inspired by manifold learning (Roweis and Saul, 2000; Zhang and Zha, 2004) and the local sub-behavior viewpoint (Han et al., 2024), we construct the atlas offline by clustering the data and fitting a low-dimensional subspace to each cluster. Online, the controller selects the subspace that best matches recent measurements and uses it to formulate and solve a predictive control problem. While Eysenbach et al. (2019) plan over stored observations using learned distances, we plan over behavioral charts using observed transitions and subspace distances to generate intermediate tracking references. Although Atlas-DeePC and BDPC use local behavioral representations, they differ in their online control and reference planning formulations. Appendix A provides a detailed comparison. The main contributions of this work are

(i) We propose Atlas-DeePC, a predictive control framework that combines local low-dimensional representations of input-output trajectories of a nonlinear system with online chart selection.

(ii) We develop a graph-guided reference planner that uses transitions observed in offline data to generate intermediate tracking references, guiding the controller through nearby charts toward targets beyond the operating region represented by the current chart.

(iii) We validate Atlas-DeePC on planar robotic reaching and autonomous racing tasks. The reaching benchmark demonstrates higher success rates and substantially lower online computation times than the compared data-selection-based predictive controllers.

The rest of the paper is organized as follows. Section 2 formulates the problem and reviews DeePC. Section 3 introduces Atlas-DeePC. Section 4 presents the planar robotic manipulator benchmark. Section 5 introduces graph-guided reference planning. Section 6 evaluates reference planning and compares Atlas-DeePC with existing methods. Section 7 concludes the paper. Appendix A extends the discussion of related work. Appendix B reports additional manipulator experiments, and Appendix C presents the autonomous vehicle example.

Notation. Let $\mathbb{Z}_{\geq 0}, \mathbb{Z}_{>0}$ denote the sets of nonnegative and positive integers, respectively. For a signal $s\colon \mathbb{Z}_{\geq 0} \to \mathbb{R}^q$ and indices $i, j \in \mathbb{Z}_{\geq 0}$ with $i \leq j$, we define $s_{[i,j]} \coloneqq (s_i, s_{i+1}, \ldots, s_j) \in \mathbb{R}^{q(j-i+1)}$. For sets X and Y, let Y^X denote the set of all maps from X to Y. For a matrix $A = [a_{ij}] \in \mathbb{R}^{r\times s}$ and index sets $\mathcal{I} \subseteq \{1, \ldots, r\}, \mathcal{J} \subseteq \{1, \ldots, s\}$, $A(\mathcal{I}, \mathcal{J})$ denotes the submatrix consisting of the rows indexed by $\mathcal{I}$ and the columns indexed by $\mathcal{J}$, arranged in increasing index order, and $A(\mathcal{I}, :)$ and $A(:, \mathcal{J})$ denote row and column submatrices, respectively. If A is a vector, we omit the colon. For consecutive indices, $a : b$ denotes the index set $\{a, a+1, \ldots, b\}$. Given a finite sequence $s_{[0,M-1]}$ and a depth $1 \leq L \leq M$, its block Hankel matrix is $\mathcal{H}_L(s_{[0,M-1]}) \coloneqq \begin{bmatrix} s_{[0,L-1]} & s_{[1,L]} & \cdots & s_{[M-L,M-1]} \end{bmatrix} \in \mathbb{R}^{qL\times(M-L+1)}$. We write I_n for the $n \times n$ identity matrix and $\mathbf{1}_n$ for the vector of n ones. The notation $\operatorname{diag}(A_1, \ldots, A_n)$ denotes the block-diagonal matrix with scalar or matrix blocks $A_1, \ldots, A_n$.

2. Problem Formulation

A *dynamical system* is a 3-tuple $(\mathbb{Z}_{\geq 0}, \mathbb{R}^q, \mathcal{B})$, where $\mathcal{B} \subseteq (\mathbb{R}^q)^{\mathbb{Z}_{\geq 0}}$ is the *behavior* (Polderman and Willems, 1998). Each $w \in \mathcal{B}$ is called a *trajectory*. The length-L *restricted behavior* is defined by $\mathcal{B}_L := \{w_{[0,L-1]} : w \in \mathcal{B}\}$. Consider the state-space system

$$x_{t+1} = f(x_t, u_t), \quad y_t = h(x_t, u_t), \tag{1}$$

where $f : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n, h : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^p$ and $x_t \in \mathbb{R}^n, u_t \in \mathbb{R}^m, y_t \in \mathbb{R}^p$ denote the state, input, output, respectively. We set $q = m + p$ and define $w_t = (u_t, y_t) \in \mathbb{R}^q$. The length-$L$ restricted behavior of (1) is defined by

$$\mathcal{B}_L = \left\{ \left(w_{[0,L-1]}\right) \in \mathbb{R}^{(m+p)L} \mid \exists x_{[0,L]} \text{ such that (1) holds for all } t \in [0, L-1] \right\}.$$

In the case that (1) is LTI of order n, its restricted behavior $\mathcal{B}_L$ is a linear subspace of $\mathbb{R}^{(m+p)L}$ with $\dim \mathcal{B}_L = mL + n$ for large enough L (Willems et al., 2005). A basis of $\mathcal{B}_L$ can be computed from data, without identifying the system parameters (Markovsky and Dörfler, 2023). Let $w_{[0,T-1]}$ be a trajectory of (1) and let $H = [w_{[0,L-1]} \cdots w_{[T-L,T-1]}]$, then $\operatorname{rank} H = mL + n$ if and only if $\operatorname{im} H = \mathcal{B}_L$. This data-based representation of a behavior underpins DeePC (Coulson et al., 2019).

Data-enabled predictive control. Let T_{ini} and T_{f} be positive integers and let $L = T_{\text{ini}} + T_{\text{f}}$. Define P to be the permutation matrix such that $Pw_{[0,L-1]} = \left[u_{[0,T_{\text{ini}}-1]}^\top y_{[0,T_{\text{ini}}-1]}^\top u_{[T_{\text{ini}},L-1]}^\top y_{[T_{\text{ini}},L-1]}^\top\right]^\top$. At time t, the sequences $u_{[t-T_{\text{ini}},t-1]}$ and $y_{[t-T_{\text{ini}},t-1]}$ are the most recent T_{ini} applied inputs and measured outputs, respectively. Let $u^{\text{f}} \in \mathbb{R}^{mT_{\text{f}}}, y^{\text{f}} \in \mathbb{R}^{pT_{\text{f}}}$ denote the future input and output sequences. Let $y^{\text{ref}} \in \mathbb{R}^{pT_{\text{f}}}$ be an output reference, Q be a positive semidefinite matrix and R be a positive definite matrix. The DeePC optimal control problem is

$$\begin{aligned} \min_{g,\sigma,u^{\text{f}},y^{\text{f}}} \quad & \left(y^{\text{f}} - y^{\text{ref}}\right)^\top Q \left(y^{\text{f}} - y^{\text{ref}}\right) + \left(u^{\text{f}}\right)^\top R u^{\text{f}} + \lambda_{\text{ini}} \|\sigma\|_2^2 \\ \text{s.t.} \quad & PHg = \begin{bmatrix} u_{[t-T_{\text{ini}},t-1]} \\ y_{[t-T_{\text{ini}},t-1]} + \sigma \\ u^{\text{f}} \\ y^{\text{f}} \end{bmatrix}, \quad u^{\text{f}} \in \mathcal{U}, \quad y^{\text{f}} \in \mathcal{Y}, \end{aligned} \tag{2}$$

where $\mathcal{U}, \mathcal{Y}$ are convex constraint sets, $\lambda_{\text{ini}} \geq 0$, and $u^{\text{f}}, y^{\text{f}}$ are decision variables representing the future input-output trajectory prediction. Let $u_t^{\text{f},*} \in \mathbb{R}^{mT_{\text{f}}}$ denote the optimal input sequence obtained from solving (2) at time t. Applying the first optimal input $u_t := u_t^{\text{f},*}(1 : m)$ and repeating the optimization at each time step defines the DeePC policy $u_t = \pi_H(w_{[t-T_{\text{ini}},t-1]}, y^{\text{ref}})$. Since the policy π_H depends entirely on the subspace $\operatorname{im} H$ (Zhang et al., 2023; Jin and Coulson, 2026), we also write π_H as $\pi_{\operatorname{im} H}$. When $\operatorname{im} H = \mathcal{B}_L$, the trajectory constraint in (2) ensures that $P^\top(u_{\text{ini}}, y_{\text{ini}} + \sigma, u^{\text{f}}, y^{\text{f}}) \in \mathcal{B}_L$. However, for nonlinear systems, $\mathcal{B}_L$ need not be a linear subspace. Consequently, $\operatorname{im} H$ may contain trajectories that cannot be generated by (1) and some trajectories of (1) may not be generated by $\operatorname{im} H$. The manifold viewpoint of nonlinear trajectories (Hauser and Meyer, 1998; Yan et al., 2026; Berberich et al., 2022b; Bayer, 2026) and trajectory clustering into local sub-behaviors (Han et al., 2024, 2025, 2026) motivate approximating $\mathcal{B}_L$ locally by low-dimensional linear subspaces using tools from manifold learning (Roweis and Saul, 2000; Zhang and Zha, 2004). We formulate the following learning and control problem.

Problem 1 *Given a collection of trajectories* $\{w^{(j)}_{[0,N_j-1]}\}_{j=1}^{N}$ *from* (1)*, we wish to: (i) learn* K *linear subspaces* $\{\mathcal{S}_k\}_{k=1}^{K}$ *that locally approximate* $\mathcal{B}_L$*; (ii) design a selection rule that chooses a subspace* $\mathcal{S}_{k_t}$ *at each time* t *based on online measured trajectory data; and (iii) design a predictive controller* $\pi_{\mathcal{S}_{k_t}}$ *that tracks an output reference while satisfying input and output constraints.*

3. Learning and control with behavioral atlases

This section develops Atlas-DeePC. Its modular structure consists of three steps: (i) clustering offline trajectories, (ii) constructing the behavioral atlas by fitting a linear subspace to each cluster, and (iii) online chart selection and predictive control.

Algorithm 1: Atlas-DeePC

Input: Offline data matrix H; feature index set $\mathcal{R}$; chart ranks $\{d_k\}_{k=1}^{K}$; measured trajectory $w_{[t-L_{\mathrm{sel}},t-1]}$; reference $y^{\mathrm{ref}} \in (\mathbb{R}^p)^{\mathbb{Z}_{\geq 0}}$

1: **Construct the atlas:** Cluster $H(\mathcal{R},:)$ to obtain index sets $\{\mathcal{I}_k\}_{k=1}^{K}$. Compute the SVD $H(:,\mathcal{I}_k) = P_k\Sigma_k V_k^\top$ and set $\Phi_k = P_k(:,1:d_k)$

2: **Select a behavioral chart:** Observe the most recent trajectory $w_{[t-L_{\mathrm{sel}},t-1]}$ and select k_t using (5)

3: **Apply the local policy:**

$$u_t = \pi_{\mathcal{S}_{k_t}}(w_{[t-T_{\mathrm{ini}},t-1]}, y^{\mathrm{ref}}_{[t,t+T_{\mathrm{f}}-1]})$$

4: **Update:** Set $t \leftarrow t+1$

5: **Repeat from line 2.**

Behavioral atlas. Consider system (1) with restricted behavior $\mathcal{B}_L$, where $L = T_{\mathrm{ini}} + T_{\mathrm{f}}$. For each $j \in [1, N]$, let $w^{(j)}_{[0,N_j-1]}$ denote a trajectory collected from (1). We refer to the jth data collection experiment as *episode* j. Let $T_j = N_j - L + 1$. We construct the Hankel data matrix of episode j as $H^{(j)} := \mathcal{H}_L\left(w^{(j)}_{[0,N_j-1]}\right) \in \mathbb{R}^{(m+p)L\times T_j}$. Here we use the permutation matrix P to be consistent to DeePC (2) row partition. Concatenating these gives the mosaic Hankel data matrix (Van Waarde et al., 2020)

$$H := \begin{bmatrix} H^{(1)} & \cdots & H^{(N)} \end{bmatrix} =: [h_1 \ \cdots \ h_T] \in \mathbb{R}^{(m+p)L\times T}, \qquad T := \sum_{j=1}^{N} T_j. \tag{3}$$

Let $\mathcal{R}$ be an index set with $|\mathcal{R}| = r$ and define $Z := H(\mathcal{R},:) = [z_1 \ \cdots \ z_T] \in \mathbb{R}^{r\times T}$. The vectors z_i are called *feature vectors* since they contain the relevant components of the input-output trajectories used for clustering (see Section 4). In the next step, a data clustering method constructs a collection of $K > 0$ nonempty sets of indices $\{\mathcal{I}_1, \ldots, \mathcal{I}_K\}$ based on the dataset $\{z_i\}_{i=1}^{T}$, where $\mathcal{I}_k \subseteq \{1, \ldots, T\}$. For each $k = 1, \ldots, K$, we define the local data matrix $H^{\mathcal{I}_k} := H(:,\mathcal{I}_k)$. The collection $\left\{H^{\mathcal{I}_1}, \ldots, H^{\mathcal{I}_K}\right\}$ therefore organizes the offline trajectories into local datasets. We abuse the terminology of charts and atlases by analogy with differential geometry. A *chart* provides Euclidean coordinates for a neighborhood on a manifold, and an *atlas* is a collection of charts covering the manifold (Lee, 2012). We construct a low-dimensional linear subspace $\mathcal{S}_k$ from $H^{\mathcal{I}_k}$, which we call the kth *behavioral chart*. For convenience, we also refer to $H^{\mathcal{I}_k}$ as the kth behavioral chart when the distinction between the data matrix and its subspace representation is clear from context. These charts form the *behavioral atlas* $\{\mathcal{S}_1, \ldots, \mathcal{S}_K\}$. For computation, we represent $\mathcal{S}_k$ by an orthonormal basis matrix Φ_k such that $\operatorname{im}\Phi_k = \mathcal{S}_k$. Affine charts can also be constructed by fitting linear subspaces to centered local data and adding the corresponding offsets (Padoan et al., 2025). Atlas-DeePC is modular and accommodates various clustering methods, such as K-means (MacQueen, 1967) or K-subspaces (Bradley and Mangasarian, 2000). We use K-means in the numerical examples in Section 4, as it provides a simple implementation and yields strong performance. A systematic comparison of clustering methods is left for future work. Specifically,

we partition the feature vectors $\{z_i\}_{i=1}^{T} \subset \mathbb{R}^r$ into K clusters. The cluster *label* for z_i is denoted $\ell_i \in [1, K]$ and the *centroid* of the kth cluster is denoted $c_k \in \mathbb{R}^r$. They are obtained by solving

$$\min_{\ell_1,\dots,\ell_T,\, c_1,\dots,c_K} \sum_{i=1}^{T} \|z_i - c_{\ell_i}\|_2^2. \tag{4}$$

The resulting index sets $\mathcal{I}_k := \{i \in \{1,\dots,T\} : \ell_i = k\}$ form a disjoint partition of $\{1,\dots,T\}$. For computational efficiency, we approximately solve (4) using mini-batch K-means (Sculley, 2010) with K-means++ initialization (Arthur et al., 2007). For each $H^{\mathcal{I}_k}$, we compute its singular value decomposition $H^{\mathcal{I}_k} = P_k \Sigma_k V_k^\top$. Let $d_k \in [1, \operatorname{rank} H^{\mathcal{I}_k}]$ be a prescribed rank, let $\Phi_k := P_k(:, 1:d_k)$ and define the kth behavioral chart as $\mathcal{S}_k = \operatorname{im} \Phi_k$.

Online chart selection and predictive control. Let $L_{\text{sel}} \le L$ denote the number of recent samples used for chart selection. At time step t, let $w_{[t-L_{\text{sel}},t-1]} \in \mathbb{R}^{(m+p)L_{\text{sel}}}$ denote the most recent length-L_{sel} trajectory and define $\mathcal{S}_k^{\text{sel}} := \operatorname{im}\left(\Phi_k(1:(m+p)L_{\text{sel}}, :)\right)$. We select the chart with the smallest projection residual, i.e.,

$$k_t \in \operatorname*{argmin}_{k \in \{1,\dots,K\}} \left\| \left(I - \Pi_{\mathcal{S}_k^{\text{sel}}}\right) w_{[t-L_{\text{sel}},t-1]} \right\|_2, \tag{5}$$

where $\Pi_{\mathcal{S}_k^{\text{sel}}}$ is the orthogonal projector onto $\mathcal{S}_k^{\text{sel}}$. If multiple charts attain the minimum residual, we select the chart with the smallest index. Given a reference output sequence $y^{\text{ref}} \in (\mathbb{R}^p)^{\mathbb{Z}_{\ge 0}}$, Atlas-DeePC (Algorithm 1) yields the control policy

$$u_t = \pi_{\mathcal{S}_{k_t}}\left(w_{[t-T_{\text{ini}},t-1]}, y^{\text{ref}}_{[t,t+T_{\text{f}}-1]}\right). \tag{6}$$

4. Simulation: Planar Robotic Manipulator

We evaluate Atlas-DeePC on a planar robotic arm tasked with moving its fingertip to a target position, with an additional F1TENTH (O'Kelly et al., 2020) autonomous vehicle experiment reported in Appendix C. We first specify the atlas construction by K-means used in the experiments. All experiments were run on an Apple M1 Pro with an 8-core CPU and 16 GB of unified memory.

Task. We use the Gymnasium (Towers et al., 2025) `Reacher-v5` environment, a MuJoCo-based simulation (Todorov et al., 2012) of a two-link planar arm tasked with moving its fingertip to a target position, with sampling period $\Delta t = 0.02$ s. Each simulation runs for $T_{\text{sim}} = 300$ steps and starts with an environment reset that randomizes the initial joint angles, joint velocities, and target position. The control input $u_t \in [-1, 1]^2$ consists of torque commands applied to the two revolute joints. Let $q_t(1)$ and $q_t(2)$ denote the joint angles at time t, and let $p_t = (p_t^x, p_t^y) \in \mathbb{R}^2$ denote the position of the fingertip at time t. The default Gymnasium observation is defined as

$$y_t := \left(\cos q_t(1), \cos q_t(2), \sin q_t(1), \sin q_t(2), \dot{q}_t(1), \dot{q}_t(2), p_t^x, p_t^y\right) \in \mathbb{R}^8.$$

Given a target position $p^{\text{goal}} \in \mathbb{R}^2$, the task is to drive the fingertip toward p^{goal} over T_{sim} control steps. Let $e_t := \|p_t - p^{\text{goal}}\|_2$ and $\operatorname*{mean}_{t_1 \le t \le t_2} e_t := \frac{1}{t_2 - t_1 + 1} \sum_{t=t_1}^{t_2} e_t$. We call a simulation successful

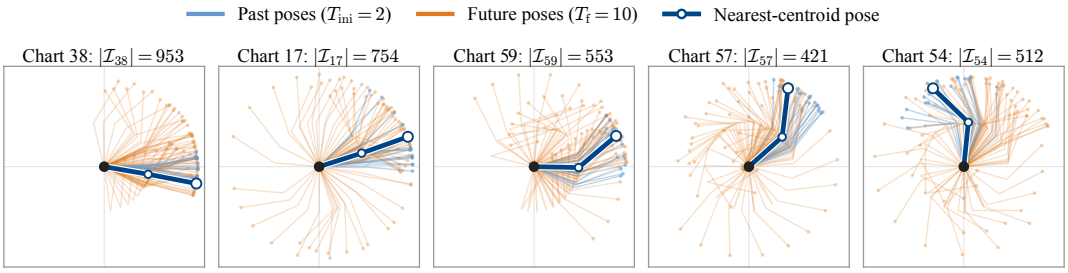


Figure 1: Sampled arm configurations from each of 5 behavioral charts. For visual clarity, only a subset of the trajectories assigned to each chart is displayed. Each sampled trajectory contains $T_{\text{ini}} = 2$ past configurations (blue) and $T_f = 10$ future configurations (orange). The bold blue arm shows the configuration corresponding to $h_{i_k^*}\big((m+p)(T_{\text{ini}}-1)+m+1:(m+p)T_{\text{ini}}\big)$, where $i_k^* \in \underset{i\in\mathcal{I}_k}{\operatorname{argmin}} \|z_i - c_k\|_2$, i.e., the arm pose closest to the centroid in clustering coordinates.

if the mean fingertip error is below $10\,\text{mm}$ over both the final 30 and final 100 time steps, and the error remains below 20 mm throughout the final 100 time steps, i.e., if

$$\underset{T_{\text{sim}}-\tau\le t\le T_{\text{sim}}-1}{\text{mean}} e_t < 10\,\text{mm} \quad \text{for } \tau \in \{30, 100\} \text{ and } \quad \underset{T_{\text{sim}}-100\le t\le T_{\text{sim}}-1}{\max} e_t < 20\,\text{mm}.$$

The 100-step mean penalizes simulations in which the fingertip approaches the target only near the end of the simulation, while the 30-step mean checks accuracy near the end of the simulation.

Behavioral atlas. We collect trajectories from $N = 200$ independent episodes, each of length $N_j = 200$. The inputs are sampled uniformly from $[-1, 1]^2$. With $T_{\text{ini}} = 2$, $T_{\text{f}} = 10$, $m = 2$, and $p = 8$, the resulting mosaic Hankel matrix (3) is $H \in \mathbb{R}^{120\times 37800}$. For atlas construction, we set the feature matrix $Z = H(\mathcal{R}, :) \in \mathbb{R}^{4\times T}$, where $\mathcal{R} \coloneqq \{(m+p)(T_{\text{ini}}-1)+m+j \mid j \in \{1,2,3,4\}\}$, i.e., the feature vectors consist of the first four components of the T_{ini}-th output of each trajectory h_i in the Hankel matrix H. The feature vectors therefore specify a configuration of the arm. We cluster Z using mini-batch K-means with $K = 64$. For each cluster k, we set $d_k = d = 53$ and construct a behavioral chart $\mathcal{S}_k = \operatorname{im}\Phi_k$ from $H^{\mathcal{I}_k}$. The effects of chart rank d and chart count K are studied in Appendix B.1. Figure 1 visualizes the arm configurations associated with five charts.

Controller settings. The task specifies only the target position $p^{\text{goal}} \in \mathbb{R}^2$, whereas the controller tracks an output reference in $\mathbb{R}^p$. We search for offline arm configurations with fingertip position closest to p^{goal}, which is formulated as

$$(j^*, t^*) \in \underset{j\in[1,N],\, t\in[0,N_j-1]}{\operatorname{argmin}} \left\| \left(y_t^{(j)}(7), y_t^{(j)}(8)\right) - p^{\text{goal}} \right\|_2.$$

We set $y_t^{\text{ref}} = \left(y_{t^*}^{(j^*)}(1), \ldots, y_{t^*}^{(j^*)}(4), 0, 0, y_{t^*}^{(j^*)}(7), y_{t^*}^{(j^*)}(8)\right)$ for all $t \in [0, T_{\text{sim}} - 1]$. The cost weights in (2) are $Q = I_{T_{\text{f}}} \otimes \operatorname{diag}(5\times 10^3 I_4, 20 I_2, 10^5 I_2)$ and $R = 10^{-4} I_{mT_{\text{f}}}$, with $\lambda_{\text{ini}} = 10^5$, where $\otimes$ denotes the Kronecker product. We impose the input constraint $\mathcal{U} = [-1, 1]^{mT_{\text{f}}}$ and no explicit output constraints. The quadratic programs are solved using Clarabel (Goulart and Chen, 2024). We select the number of clusters K and the chart rank d by evaluating the control performance over 100 simulations and choosing the best-performing hyperparameters (see Appendix B.1 for a more detailed analysis of the effects on the performance). The performance of Atlas-DeePC with the selected hyperparameters is then evaluated over 500 test simulations.

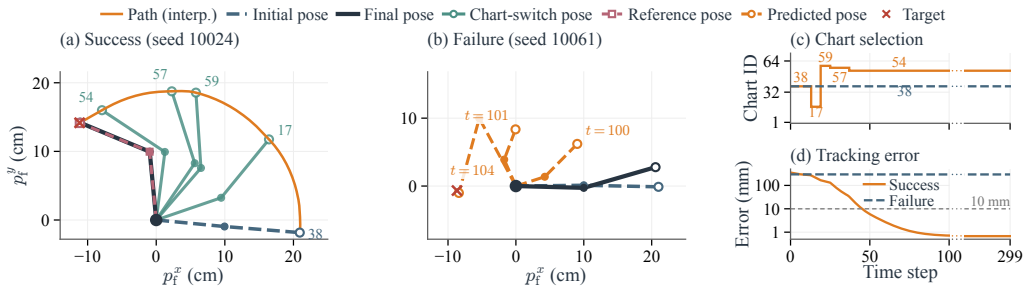


Figure 2: Atlas-DeePC simulations. (a) Fingertip path and arm poses at chart switches (green), labeled by the newly selected chart indices. (b) Failed simulation, with predicted poses (dashed orange line) from a single prediction horizon computed at $t = 100$. Predicted joint angles and fingertip positions need not be kinematically consistent. The dashed poses show the predicted first-joint direction and fingertip position, without using the predicted second-joint angle. (c) Selected chart indices. (d) Fingertip errors e_t on a logarithmic scale.

Results. Figure 2 presents one successful and one failed simulation. In the successful case, Atlas-DeePC drives the fingertip to the target, with $\underset{T_{\text{sim}}-30 \le t \le T_{\text{sim}}-1}{\text{mean}} e_t < 1\text{mm}$ (Figure 2 (d)). In the failed case, the controller remains in one chart despite predicted poses approaching the target (Figure 2(b)). This motivates a motion planning module that constructs intermediate references to guide the controller through behavioral charts toward the target. A statistical evaluation over 500 test simulations is presented in Section 6.

5. Graph-Guided Reference Planning

To guide the controller toward goals beyond the current operating region (i.e., towards trajectory references that are not in the selected behavioral charts), we select intermediate references using a weighted directed graph of observed chart transitions.

Transition graph construction. We denote the set of successor pairs in the offline data by

$$\mathcal{T} := \Big\{ (i, i+1) : i \in [1, T-1], h_i \text{ and } h_{i+1} \text{ are columns of } H^{(j)} \text{ for some } j \in [1, N] \Big\}.$$

Using the chart labels $\{\ell_i\}_{i=1}^{T}$ from (4), define the transition count from chart k to chart k' as

$$n_{kk'} := \big| \{ (i, i+1) \in \mathcal{T} \mid \ell_i = k,\ \ell_{i+1} = k' \} \big|. \tag{7}$$

We define the directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with node set $\mathcal{V} = \{1, \dots, K\}$ and edge set $\mathcal{E} := \{(k, k') : k \neq k',\ n_{kk'} \geq n_{\min}\}$, where the user-defined threshold $n_{\min} \geq 1$ excludes rarely observed transitions. Each edge $(k, k') \in \mathcal{E}$ is assigned the weight $\omega_{kk'} := d_\kappa(\mathcal{S}_k, \mathcal{S}_{k'}) / \sqrt{n_{kk'}}$, where d_κ is the chordal distance (Ye and Lim, 2016; Padoan and Coulson, 2025) defined by $d_\kappa(\mathcal{S}_k, \mathcal{S}_{k'}) = \frac{1}{\sqrt{2}} \|\Phi_k \Phi_k^\top - \Phi_{k'} \Phi_{k'}^\top\|_F$, measuring the distance between subspaces $\mathcal{S}_k$ and $\mathcal{S}_{k'}$. The edge weight is smaller for frequently observed transitions between similar behavioral subspaces.

Online reference planning. Let k_t denote the chart index selected at time t. Let $y_t^{\text{term}} \in \mathbb{R}^p$ and k_t^{term} denote an application-dependent terminal reference and chart index. Let $\mathcal{P}(k_t, k_t^{\text{term}})$ denote the set of paths from k_t to k_t^{term} in $\mathcal{G}$, where a *path* ρ is a finite ordered tuple of distinct nodes $\rho = (v_0, \dots, v_M)$ where each $v_j \in \mathcal{V}$ and $v_0 = k_t$, $v_M = k_t^{\text{term}}$, and $(v_{j-1}, v_j) \in \mathcal{E}$ for each

$j \in [1, M]$. We denote the edge set of the path ρ by $\mathcal{E}(\rho) := \{(v_{j-1}, v_j) : j \in [1, M]\}$. Whenever $\mathcal{P}(k_t, k_t^{\text{term}})$ is nonempty, we compute a shortest path

$$\rho_t^* := (k_t^0, k_t^1, \ldots, k_t^{M_t}) \in \operatorname*{argmin}_{\rho \in \mathcal{P}(k_t, k_t^{\text{term}})} \sum_{(i,j)\in\mathcal{E}(\rho)} \omega_{ij}, \quad k_t^0 := k_t, \quad k_t^{M_t} := k_t^{\text{term}} \tag{8}$$

using Dijkstra's algorithm (Dijkstra, 1959). The corresponding minimum path cost is denoted by

$$d_{\mathcal{G}}(k_t, k_t^{\text{term}}) := \sum_{(i,j)\in\mathcal{E}(\rho_t^*)} \omega_{ij}, \tag{9}$$

with cost zero when $k_t = k_t^{\text{term}}$ and $+\infty$ when no path exists. If $k_t \neq k_t^{\text{term}}$, we select an intermediate output $y_t^{\text{int}} \in \mathbb{R}^p$ from the next chart $H\left(:, \mathcal{I}_{k_t^1}\right)$ in ρ_t^* using an application-dependent rule (see Section 6), and set $y_{[t,t+T_{\text{f}}-1]}^{\text{ref}} = \mathbf{1}_{T_{\text{f}}} \otimes y_t^{\text{int}}$. If $k_t = k_t^{\text{term}}$, we set $y_{[t,t+T_{\text{f}}-1]}^{\text{ref}} = \mathbf{1}_{T_{\text{f}}} \otimes y_t^{\text{term}}$. As new measurements become available, we repeat the procedure using the newly selected chart $\mathcal{S}_{k_{t+1}}$ from (5), which need not coincide with the next chart $\mathcal{S}_{k_t^1}$ on the shortest path.

6. Simulation: Reference Planning and Algorithm Comparison

Reference selection for Reacher. We now describe how terminal and intermediate references are selected for the Reacher task introduced in Section 4. We set $n_{\min} = 3$ when constructing the chart-transition graph, so each edge corresponds to at least three observed transitions in the offline data. Let $\bar{y}_i := h_i\big((m+p)(T_{\text{ini}}-1)+m+1 : (m+p)T_{\text{ini}}\big) \in \mathbb{R}^p$, which represents an offline arm pose. For each chart $\mathcal{S}_k$, we compute and store

$$i_k^* \in \operatorname*{argmin}_{i\in\mathcal{I}_k} \|z_i - c_k\|_2^2. \tag{10}$$

Thus, $\bar{y}_k^* := \bar{y}_{i_k^*}$ is an observed arm configuration from cluster k whose feature vector is closest to the centroid c_k. Since the centroid c_k averages the feature vectors in cluster k and need not correspond to an observed trajectory, we use $\bar{y}_{i_k^*}$ as the representative configuration of chart $\mathcal{S}_k$ (Figure 1). Different arm poses can place the fingertip near the same target, but may differ in their reachability from the current chart. For a given target p^{goal}, we choose ten offline outputs $\bar{y}_i$ whose fingertip positions $(\bar{y}_i(7), \bar{y}_i(8))$ are closest to the target p^{goal}. Their indices form the candidate set $\mathcal{C}$ defined by

$$\mathcal{C} \in \operatorname*{argmin}_{\substack{\mathcal{J}\subseteq\{1,\ldots,T\} \\ |\mathcal{J}|=10}} \sum_{i\in\mathcal{J}} \left\| \big(\bar{y}_i(7), \bar{y}_i(8)\big) - p^{\text{goal}} \right\|_2.$$

Each candidate indexed by $i \in \mathcal{C}$ is associated with chart $\mathcal{S}_{\ell_i}$, where ℓ_i is its cluster label from (4). At each time t, we select the candidate whose chart has the minimum shortest-path cost from the current chart, i.e.,

$$i_t^{\text{term}} \in \operatorname*{argmin}_{i\in\mathcal{C}} d_{\mathcal{G}}(k_t, \ell_i), \qquad k_t^{\text{term}} := \ell_{i_t^{\text{term}}}.$$

If multiple candidates attain the minimum path cost, we select the one with the smallest fingertip distance to p^{goal}. If these distances are also equal, we choose the smallest index. The terminal reference is the selected output with its joint-velocity components set to zero, i.e.,

$$y_t^{\text{term}} := \big(\bar{y}_{i_t^{\text{term}}}(1), \ldots, \bar{y}_{i_t^{\text{term}}}(4), 0, 0, \bar{y}_{i_t^{\text{term}}}(7), \bar{y}_{i_t^{\text{term}}}(8)\big).$$

Output dimension	$p = 8$		$p = 10$	
Reference Planning	No	Yes	No	Yes
Chart count K	64	96	80	64
Selected rank	53	57	64	62
Success rate	90.0%	94.2%	83.8%	**98.8%**
Median error (mm)	1.81	1.86	**1.51**	1.92
90th percentile (mm)	10.05	4.61	232.29	**3.81**
95th percentile (mm)	218.27	12.12	296.36	**4.63**
99th percentile (mm)	299.30	48.23	371.37	**10.62**
Median online computation time (ms/step)				
Chart selection	0.16	0.21	0.22	0.19
Graph guidance	—	0.02	—	0.02
QP solve	19.81	19.65	22.77	22.41
Total	20.01	19.92	23.03	22.65

Table 1: Atlas-DeePC performance and computational cost on 500 test simulations comparing output dimensions and graph guidance. Error statistics summarize final-30 mean fingertip errors. Chart selection measures the time spent computing projection residuals and selecting the behavioral chart.

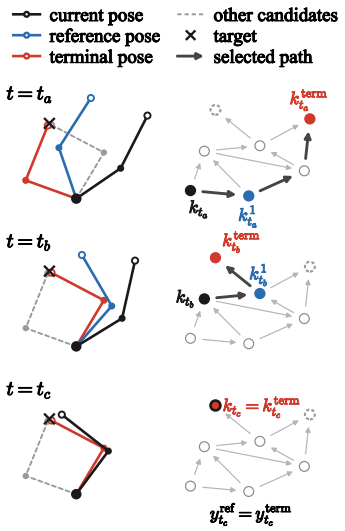


Figure 3: Graph-guided reference selection. Intermediate (blue) and terminal (red) poses are updated as the current chart changes ($t_a \to t_b$).

For the shortest path $(k_t^0, k_t^1, \ldots, k_t^{M_t})$ given by (8), we construct the intermediate output from the representative configuration of the next chart $\mathcal{S}_{k_t^1}$ given by (10):

$$y_t^{\text{int}} := \left(\bar{y}^*_{k_t^1}(1), \ldots, \bar{y}^*_{k_t^1}(4), 0, 0, \bar{y}^*_{k_t^1}(7), \bar{y}^*_{k_t^1}(8)\right).$$

The reference supplied to Algorithm 1 is $y^{\text{ref}}_{[t,t+T_{\text{f}}-1]} = \mathbf{1}_{T_{\text{f}}} \otimes y_t^{\text{term}}$ if $k_t = k_t^{\text{term}}$ or $\min_{0\le\tau\le t} e_\tau < 70\,\text{mm}$, and $y^{\text{ref}}_{[t,t+T_{\text{f}}-1]} = \mathbf{1}_{T_{\text{f}}} \otimes y_t^{\text{int}}$ otherwise. We use a heuristic threshold of $70\,\text{mm}$ to switch to direct terminal-reference tracking once the fingertip comes within this distance of the target, without claiming that this threshold is optimal. Figure 3 illustrates this procedure. At t_a, the next chart on the selected shortest path provides an intermediate reference (blue) toward the terminal pose (red). At t_b, the current chart has changed, leading to a different terminal candidate and an updated intermediate reference. At t_c, the current and terminal charts coincide, so the controller tracks the terminal reference directly.

Output augmentation. Alongside graph guidance, we study whether additional output coordinates improve tracking. Motivated by observable-based Koopman prediction and control (Lian and Jones, 2021; Huang et al., 2024), we augment the 8-dimensional output with the absolute orientation of the second link by $(y_t, \cos(q_t(1)+q_t(2)), \sin(q_t(1)+q_t(2))) \in \mathbb{R}^{10}$. The augmentation is evaluated empirically. Table 1 compares Atlas-DeePC with and without graph guidance for output dimensions $p = 8$ and $p = 10$ over 500 test simulations, with chart counts and ranks selected separately based on development simulations reported in Appendix B.1. Here, "error" denotes $\operatorname{mean}_{T_{\text{sim}}-30\le t\le T_{\text{sim}}-1} e_t$. For each fixed output dimension p, reference planning improves the success rate and percentile errors. Output augmentation alone does not improve performance, but its combination with reference planning achieves the best performance. These results suggest that adding

Method	Atlas-DeePC		Select	Contextual
Reference planning	No	Yes	No	No
Coefficient dimension	53	57	200	200
Success rate	90.0%	**94.2%**	83.0%	84.0%
Median error (mm)	**1.81**	1.86	2.97	6.16
90th percentile (mm)	10.05	**4.61**	29.95	11.66
95th percentile (mm)	218.27	**12.12**	58.58	13.49
99th percentile (mm)	299.30	**48.23**	83.48	91.92
Total online computation (ms per step)				
Median	20.01	**19.92**	1330.85	79.77
95th percentile	27.36	**23.15**	3345.22	95.36
99th percentile	53.85	**43.17**	4838.75	111.84

Table 2: Algorithm comparison on 500 test simulations (seeds 10001–10500). "Select" and "Contextual" denote select-DPC and contextual-DPC, respectively. Atlas-DeePC uses $p = 8$, with $K = 64$ and $d = 53$ without guidance, and $K = 96$ and $d = 57$ with guidance. Coefficient dimension denotes the number of entries in the trajectory coefficient vector g (2). Contextual-DPC uses the same fixed terminal references as unguided Atlas-DeePC. Error statistics summarize final-30 mean fingertip errors across simulations. For Atlas-DeePC, the median online computation time is comparable to the 20 ms sampling interval, although some steps exceed this interval.

output coordinates does not necessarily improve closed-loop performance. Appendix B.2 isolates the effect of reference planning and provides additional individual simulation plots.

Comparison with existing methods. We compare Atlas-DeePC with select-DPC (Näf et al., 2025) and contextual-DPC (Beerwerth and Alrifaee, 2026). Both baselines select 200 trajectory columns from their respective offline datasets using relevance-based criteria and use these columns directly in the predictive control problem. Implementation details are provided in Appendix B.3. Our implementation of KDE-BDPC (Han et al., 2026) did not achieve successful tracking on the Reacher benchmark and is therefore excluded from the main quantitative comparison. Appendix A discusses the differences between its one-step control formulation and Atlas-DeePC. We conjecture that the one-step prediction of KDE-BDPC is not sufficient for long-term task planning required for the planar robotic task. Table 2 summarizes results over 500 test seeds. Similar to Table 1, "error" denotes $\underset{T_{\mathrm{sim}}-30\le t\le T_{\mathrm{sim}}-1}{\mathrm{mean}} e_t$. In this benchmark, Atlas-DeePC achieves higher success rates and lower median fingertip errors than both Select- and Contextual-DPC, with substantially lower online computation times. Graph-guided reference planning further improves the success rate and reduces upper-percentile errors, while maintaining a similar computational cost.

7. Conclusion

We presented Atlas-DeePC, a modular data-driven predictive control framework that constructs a behavioral atlas of low-dimensional charts from offline input-output trajectories. Online, the controller selects a chart from the behavioral atlas and applies the control policy (6). An optional graph-guided reference planner uses observed chart transitions to generate intermediate references toward the target. On the planar robotic manipulator benchmark, Atlas-DeePC achieved better performance and substantially lower online computation time than the compared methods. Future work will investigate principled choices of chart rank and chart count, approximation guarantees for nonlinear behavioral charts, and conditions for reliable closed-loop transitions between charts. We also plan to evaluate Atlas-DeePC on physical robotic platforms.

Acknowledgments

We used ChatGPT to assist with grammar checking, language editing, and code revision. The authors reviewed the resulting text and take responsibility for the final manuscript and implementation.

References

Maximilian Adang, Timothy Chen, Lars Osterberg, Aiden Swann, and Mac Schwager. Training-free behavior cloning. *arXiv preprint arXiv:2609.30134*, 2026.

Mohammad Alsalti, Manuel Barkey, Victor G. Lopez, and Matthias A. Müller. Robust and efficient data-driven predictive control. *Automatica*, 191:113108, September 2026.

David Arthur, Sergei Vassilvitskii, et al. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, volume 7, pages 1027–1035, 2007.

Arda Bayer. Trajectory manifolds for nonlinear data-enabled predictive control. *arXiv preprint arXiv:2609.19079*, 2026.

Julius Beerwerth and Bassam Alrifaee. Less is more: Contextual sampling for nonlinear data-driven predictive control. In *2026 European Control Conference (ECC)*, pages 1953–1958. IEEE, 2026.

Alberto Bemporad. A piecewise linear regression and classification algorithm with application to learning and model predictive control of hybrid systems. *IEEE Transactions on Automatic Control*, 68(6):3194–3209, 2023.

Julian Berberich and Frank Allgöwer. An overview of systems-theoretic guarantees in data-driven model predictive control. *Annual Review of Control, Robotics, and Autonomous Systems*, 8(1): 77–100, 2025.

Julian Berberich, Johannes Köhler, Matthias A. Müller, and Frank Allgöwer. Data-driven model predictive control with stability and robustness guarantees. *IEEE Transactions on Automatic Control*, 66(4):1702–1717, 2021. doi: 10.1109/TAC.2020.3000182.

Julian Berberich, Johannes Köhler, Matthias A Müller, and Frank Allgöwer. Linear tracking MPC for nonlinear systems—part II: The data-driven case. *IEEE Transactions on Automatic Control*, 67(9):4406–4421, 2022a.

Julian Berberich, Johannes Köhler, Matthias A Müller, and Frank Allgöwer. Linear tracking MPC for nonlinear systems—part I: The model-based case. *IEEE Transactions on Automatic Control*, 67(9):4390–4405, 2022b.

Paul S Bradley and Olvi L Mangasarian. K-plane clustering. *Journal of Global Optimization*, 16 (1):23–32, 2000.

Steven L Brunton, Bingni W Brunton, Joshua L Proctor, and J Nathan Kutz. Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control. *PLOS ONE*, 11(2):e0150171, 2016.

Eduardo F. Camacho and Carlos Bordons. *Model Predictive Control*. Advanced Textbooks in Control and Signal Processing. Springer, London, 2 edition, 2007.

Jingshan Chen, Henrik Ebel, and Peter Eberhard. Practical insights on data-based robot control: A comparative analysis of data-enabled predictive control and model-based predictive control. *Journal of Intelligent & Robotic Systems*, 112(2):32, 2026.

Jeremy Coulson, John Lygeros, and Florian Dörfler. Data-enabled predictive control: In the shallows of the DeePC. In *2019 18th European Control Conference (ECC)*, pages 307–312. IEEE, 2019.

Jeremy Coulson, John Lygeros, and Florian Dörfler. Distributionally robust chance constrained data-enabled predictive control. *IEEE Transactions on Automatic Control*, 67(7):3289–3304, 2022.

Thomas O de Jong, Mircea Lazar, Siep Weiland, and Florian Dörfler. Scalable nonlinear DeePC: Bridging direct and indirect methods and basis reduction. *arXiv preprint arXiv:2512.14535*, 2025.

Claudio De Persis and Pietro Tesi. Formulas for data-driven control: Stabilization, optimality, and robustness. *IEEE Transactions on Automatic Control*, 65(3):909–924, 2020.

Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

Florian Dörfler, Jeremy Coulson, and Ivan Markovsky. Bridging direct and indirect data-driven control formulations via regularizations and relaxations. *IEEE Transactions on Automatic Control*, 68(2):883–897, 2023.

Ben Eysenbach, Russ R Salakhutdinov, and Sergey Levine. Search on the replay buffer: Bridging planning and reinforcement learning. *Advances in neural information processing systems*, 32, 2019.

Randall T Fawcett, Kereshmeh Afsari, Aaron D Ames, and Kaveh Akbari Hamed. Toward a data-driven template model for quadrupedal locomotion. *IEEE Robotics and Automation Letters*, 7 (3):7636–7643, 2022.

Giancarlo Ferrari-Trecate, Marco Muselli, Diego Liberati, and Manfred Morari. A clustering technique for the identification of piecewise affine systems. *Automatica*, 39(2):205–217, 2003.

Paul J. Goulart and Yuwen Chen. Clarabel: An interior-point solver for conic programs with quadratic objectives, 2024.

Margarita A Guerrero, Braghadeesh Lakshminarayanan, and Cristian R Rojas. Gain-scheduled data-enabled predictive control: A DeePC approach for nonlinear systems. *IEEE Control Systems Letters*, 9:3041–3046, 2025.

Shuangyu Han, Yitao Yan, Jie Bao, and Biao Huang. Big data-driven predictive control using multi-view clustering. In *2024 American Control Conference (ACC)*, pages 5100–5105. IEEE, 2024.

Shuangyu Han, Yitao Yan, Jie Bao, and Biao Huang. Big data-driven predictive control for nonlinear systems—a trajectory cluster-based contraction approach. *Journal of Process Control*, 152: 103474, 2025.

Shuangyu Han, Yitao Yan, Jie Bao, and Biao Huang. Big data-driven predictive control for nonlinear systems based on kernel density estimation of data trajectories. *Computers & Chemical Engineering*, page 109565, 2026.

Nathan Hatch and Byron Boots. The value of planning for infinite-horizon model predictive control. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7372–7378. IEEE, 2021.

John Hauser and David G Meyer. The trajectory manifold of a nonlinear control system. In *Proceedings of the 37th IEEE Conference on Decision and Control (Cat. No. 98CH36171)*, volume 1, pages 1034–1039. IEEE, 1998.

Linbin Huang, Jianzhe Zhen, John Lygeros, and Florian Dörfler. Quadratic regularization of data-enabled predictive control: Theory and application to power converter experiments. *IFAC-PapersOnLine*, 54(7):192–197, 2021.

Linbin Huang, Jeremy Coulson, John Lygeros, and Florian Dörfler. Decentralized data-enabled predictive control for power system oscillation damping. *IEEE Transactions on Control Systems Technology*, 30(3):1065–1077, 2022.

Linbin Huang, John Lygeros, and Florian Dörfler. Robust and kernelized data-enabled predictive control for nonlinear systems. *IEEE Transactions on Control Systems Technology*, 32(2):611–624, 2024.

Dian Jin and Jeremy Coulson. Online subspace learning on flag manifolds for system identification. *arXiv preprint arXiv:2511.06416*, 2025.

Dian Jin and Jeremy Coulson. On the sensitivity of the subspace predictor to behavioral perturbations. *IEEE Control Systems Letters*, 2026.

Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018.

Mircea Lazar. Basis-functions nonlinear data-enabled predictive control: Consistent and computationally efficient formulations. In *2024 European Control Conference (ECC)*, pages 888–893. IEEE, 2024a.

Mircea Lazar. Neural data–enabled predictive control. *IFAC-PapersOnLine*, 58(15):91–96, 2024b.

John M. Lee. *Introduction to Smooth Manifolds*. Springer, New York, 2012. ISBN 978-1-4899-9475-2 978-1-4419-9982-5.

Kejun Li, Jeeseop Kim, Xiaobin Xiong, Kaveh Akbari Hamed, Yisong Yue, and Aaron D. Ames. Data-driven predictive control for robust exoskeleton locomotion. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 162–169, 2024.

Kejun Li, Jeeseop Kim, Maxime Brunet, Marine Pétriaux, Yisong Yue, and Aaron D Ames. Hybrid data-driven predictive control for robust and reactive exoskeleton locomotion synthesis. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10077–10084. IEEE, 2025.

Yingzhao Lian and Colin N Jones. Nonlinear data-enabled prediction and control. In *Learning for Dynamics and Control*, pages 523–534. PMLR, 2021.

David J.N. Limebeer and Robin S. Sharp. Single-track vehicle modeling and control: Bicycles, motorcycles, and models. *IEEE Control Systems*, 26(5):34–61, October 2006. ISSN 1066-033X. doi: 10.1109/MCS.2006.1700044.

James B MacQueen. Some methods for classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.*, pages 281–297, 1967.

Ivan Markovsky and Florian Dörfler. Behavioral systems theory in data-driven analysis, signal processing, and control. *Annual Reviews in Control*, 52:42–64, 2021.

Ivan Markovsky and Florian Dörfler. Identifiability in the behavioral setting. *IEEE Transactions on Automatic Control*, 68(3):1667–1677, 2023.

Ivan Markovsky and Paolo Rapisarda. Data-driven simulation and control. *International Journal of Control*, 81(12):1946–1959, 2008.

Ivan Markovsky, Linbin Huang, and Florian Dorfler. Data-driven control based on the behavioral approach from theory to applications in power systems. *IEEE Control Systems Magazine*, 43(5): 28–68, 2023.

Joshua Näf, Keith Moffat, Jaap Eising, and Florian Dörfler. Choose wisely: Data-driven predictive control for nonlinear systems using online data selection. *arXiv preprint arXiv:2503.18845*, 2025.

Matthew O'Kelly, Hongrui Zheng, Dhruv Karthik, and Rahul Mangharam. F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning. *Proceedings of Machine Learning Research*, 123, 2020.

Cheng Ouyang, Moeen Ul Islam, Dong Chen, Kaixiang Zhang, Zhaojian Li, and Xiaobo Tan. Direct data-driven predictive control for a three-dimensional cable-driven soft robotic arm. In *2026 American Control Conference (ACC)*, pages 3977–3982. IEEE, 2026.

Alberto Padoan and Jeremy Coulson. Distances between finite-horizon linear behaviors. *IEEE Control Systems Letters*, 9:1772–1777, 2025.

Alberto Padoan, Jaap Eising, and Ivan Markovsky. From time series to affine systems. *arXiv preprint arXiv:2510.22089*, 2025.

Simone Paoletti, Aleksandar Lj Juloski, Giancarlo Ferrari-Trecate, and René Vidal. Identification of hybrid systems a tutorial. *European Journal of Control*, 13(2-3):242–260, 2007.

Jan Willem Polderman and Jan C. Willems. *Introduction to Mathematical Systems Theory: A Behavioral Approach*, volume 26 of *Texts in Applied Mathematics*. Springer, New York, 1998. doi: 10.1007/978-1-4757-2953-5.

Sam T Roweis and Lawrence K Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.

Wilson J Rugh and Jeff S Shamma. Research on gain scheduling. *Automatica*, 36(10):1401–1425, 2000.

András Sasfi, Jaap Eising, and Florian Dörfler. Soft projections for robust data-driven control. *arXiv preprint arXiv:2604.00900*, 2026a.

András Sasfi, Alberto Padoan, Ivan Markovsky, and Florian Dörfler. GREAT: Grassmannian recursive algorithm for tracking & online system identification. *IEEE Transactions on Automatic Control*, 2026b.

David Sculley. Web-scale k-means clustering. In *Proceedings of the 19th International Conference on World Wide Web*, pages 1177–1178, 2010.

Xu Shang and Yang Zheng. Convex approximations for a bi-level formulation of data-enabled predictive control. In *6th Annual Learning for Dynamics & Control Conference*, pages 1071–1082. PMLR, 2024.

Akshun Sharma, Zhongqi Wei, Buqing Ou, and Frederike Dümbgen. RoKON: Efficient and adaptive data-enabled predictive control for robotic manipulation. *arXiv preprint arXiv:2610.35458*, 2026.

Jicheng Shi, Yingzhao Lian, and Colin N Jones. Efficient recursive data-enabled predictive control. In *2024 European Control Conference (ECC)*, pages 880–887. IEEE, 2024.

Johannes Teutsch, Sebastian Ellmaier, Sebastian Kerz, Dirk Wollherr, and Marion Leibold. An online adaptation strategy for direct data-driven control. *IFAC-PapersOnLine*, 56(2):644–649, 2023.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.

Mark Towers, Ariel Kwiatkowski, John Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Kallinteris Andreas, Markus Krimmel, Arjun Kg, Rodrigo Perez-Vicente, et al. Gymnasium: A standard interface for reinforcement learning environments. *Advances in Neural Information Processing Systems*, 38, 2025.

Amin Vahidi-Moghaddam, Kaixiang Zhang, Xunyuan Yin, Vaibhav Srivastava, and Zhaojian Li. Online reduced-order data-enabled predictive control. *IEEE Transactions on Automation Science and Engineering*, 2025.

Henk J Van Waarde, Claudio De Persis, M Kanat Camlibel, and Pietro Tesi. Willems' fundamental lemma for state-space systems and its extension to multiple datasets. *IEEE Control Systems Letters*, 4(3):602–607, 2020.

Chris Verhoek, Roland Tóth, Sofie Haesaert, and Anne Koch. Fundamental lemma for data-driven analysis of linear parameter-varying systems. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 5040–5046. IEEE, 2021.

Huanqing Wang, Kaixiang Zhang, Kyungjoon Lee, Yu Mei, Vaibhav Srivastava, Jun Sheng, Ziyou Song, and Zhaojian Li. Velocity-form data-enabled predictive control of soft robots under unknown external payloads. *IEEE Robotics and Automation Letters*, 2026.

Jan C Willems. From time series to linear system—part I. finite dimensional linear time invariant systems. *Automatica*, 22(5):561–580, 1986.

Jan C Willems. The behavioral approach to open and interconnected systems. *IEEE Control Systems Magazine*, 27(6):46–99, 2007.

Jan C Willems, Paolo Rapisarda, Ivan Markovsky, and Bart LM De Moor. A note on persistency of excitation. *Systems & Control Letters*, 54(4):325–329, 2005.

Xiao Wu, Jiong Shen, Yiguo Li, and Kwang Y Lee. Data-driven modeling and predictive control for boiler–turbine unit using fuzzy clustering and subspace methods. *ISA Transactions*, 53(3): 699–708, 2014.

Yitao Yan, Yu Tong, Jie Bao, and Wei Wang. Behavioral systems theory meets machine learning: Control-aware learning of the intrinsic behavior from big data. *arXiv preprint arXiv:2604.13673*, 2026.

Ke Ye and Lek-Heng Lim. Schubert varieties and distances between subspaces of different dimensions. *SIAM Journal on Matrix Analysis and Applications*, 37(3):1176–1197, 2016.

Yicheng Zeng, Yuhao Huang, and Xiaobin Xiong. Reference-steering via data-driven predictive control for hyper-accurate robotic flying-hopping locomotion. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3760–3766. IEEE, 2025.

Kaixiang Zhang, Yang Zheng, Chao Shang, and Zhaojian Li. Dimension reduction for efficient data-enabled predictive control. *IEEE Control Systems Letters*, 7:3277–3282, 2023.

Zhenyue Zhang and Hongyuan Zha. Principal manifolds and nonlinear dimensionality reduction via tangent space alignment. *SIAM Journal on Scientific Computing*, 26(1):313–338, 2004.

Sebastian Zieglmeier, Mathias Hudoba de Badyn, Narada D Warakagoda, Thomas R Krogstad, and Paal Engelstad. Gain-scheduling data-enabled predictive control for nonlinear systems with linearized operating regions. *arXiv preprint arXiv:2512.02797*, 2025.

Appendix A. Extended Related Work

Behavioral systems theory and data-driven control Behavioral systems theory describes a dynamical system through its admissible trajectories, independently of a particular representation such as state space (Willems, 1986; Polderman and Willems, 1998; Willems, 2007). For controllable LTI systems, the fundamental lemma (Willems et al., 2005) gives conditions under which the set of all finite-length trajectories can be spanned by a Hankel matrix constructed from input-output data. This viewpoint provides a foundation for simulation and control directly from input-output data (Markovsky and Rapisarda, 2008; Coulson et al., 2019; Markovsky and Dörfler, 2021). Further developments address stabilization, optimality, robustness, and constraint satisfaction (De Persis and Tesi, 2020; Berberich et al., 2021; Coulson et al., 2022; Dörfler et al., 2023; Markovsky and Dörfler, 2023; Sasfi et al., 2026a). Extensions to broader system classes include a fundamental lemma for linear parameter-varying systems (Verhoek et al., 2021). Complementary work on nonlinear and hybrid systems uses data partitioning and local affine-model fitting to identify piecewise-affine dynamics (Ferrari-Trecate et al., 2003; Paoletti et al., 2007; Bemporad, 2023). Nonlinear extensions also use basis functions and learned neural features to construct data-enabled predictors (Lazar, 2024a,b). Further developments address computational efficiency through dimension reduction (Zhang et al., 2023; Shi et al., 2024; Shang and Zheng, 2024; Vahidi-Moghaddam et al., 2025; de Jong et al., 2025; Alsalti et al., 2026), online identification through subspace tracking (Sasfi et al., 2026b; Jin and Coulson, 2025), and online adaptation of data-based representations for nonlinear control (Teutsch et al., 2023). DeePC has been applied across a wide range of domains, such as power systems (Huang et al., 2021, 2022), and robotics (Fawcett et al., 2022; Li et al., 2024, 2025; Zeng et al., 2025; Adang et al., 2026; Sharma et al., 2026; Wang et al., 2026; Ouyang et al., 2026; Chen et al., 2026).

Trajectory clustering for data-driven control Clustering provides a way to organize trajectories of a nonlinear system into local representations for prediction and control. Early work uses clustering to identify piecewise-affine models from input-output data (Ferrari-Trecate et al., 2003). Wu et al. (2014) combine fuzzy operating-region memberships with subspace identification to construct predictive controllers. Gain scheduling constructs controllers for different operating conditions (Rugh and Shamma, 2000). Following this approach, gain-scheduled DeePC partitions the data into operating regions according to a prescribed scheduling variable (Guerrero et al., 2025; Zieglmeier et al., 2025). Choosing a suitable scheduling variable requires sufficient system knowledge to identify the dominant nonlinearity (Zieglmeier et al., 2025, Assumption 2). In contrast, Atlas-DeePC clusters task-dependent features of measured trajectories without requiring explicit identification of the dominant nonlinearity. More closely related big data-driven predictive control (BDPC) approaches cluster input-output trajectories into local sub-behaviors (Han et al., 2024, 2025, 2026). Their control formulations optimize a single future input-output pair conditioned on measured history (Han et al., 2024, Eq. (11)) (Han et al., 2025, Eq. (45)) (Han et al., 2026, Eq. (34)). In contrast, Atlas-DeePC follows the standard multi-step receding-horizon formulation in predictive control. At each control step, it optimizes input-output trajectories over T_{f} future steps, applies the first optimal input, and repeats the optimization using updated measurements. Unlike one-step prediction, multi-step prediction allows the controller to evaluate control actions through their predicted effects over several future steps, reducing short-sightedness associated with optimizing only immediate tracking performance (Hatch and Boots, 2021).

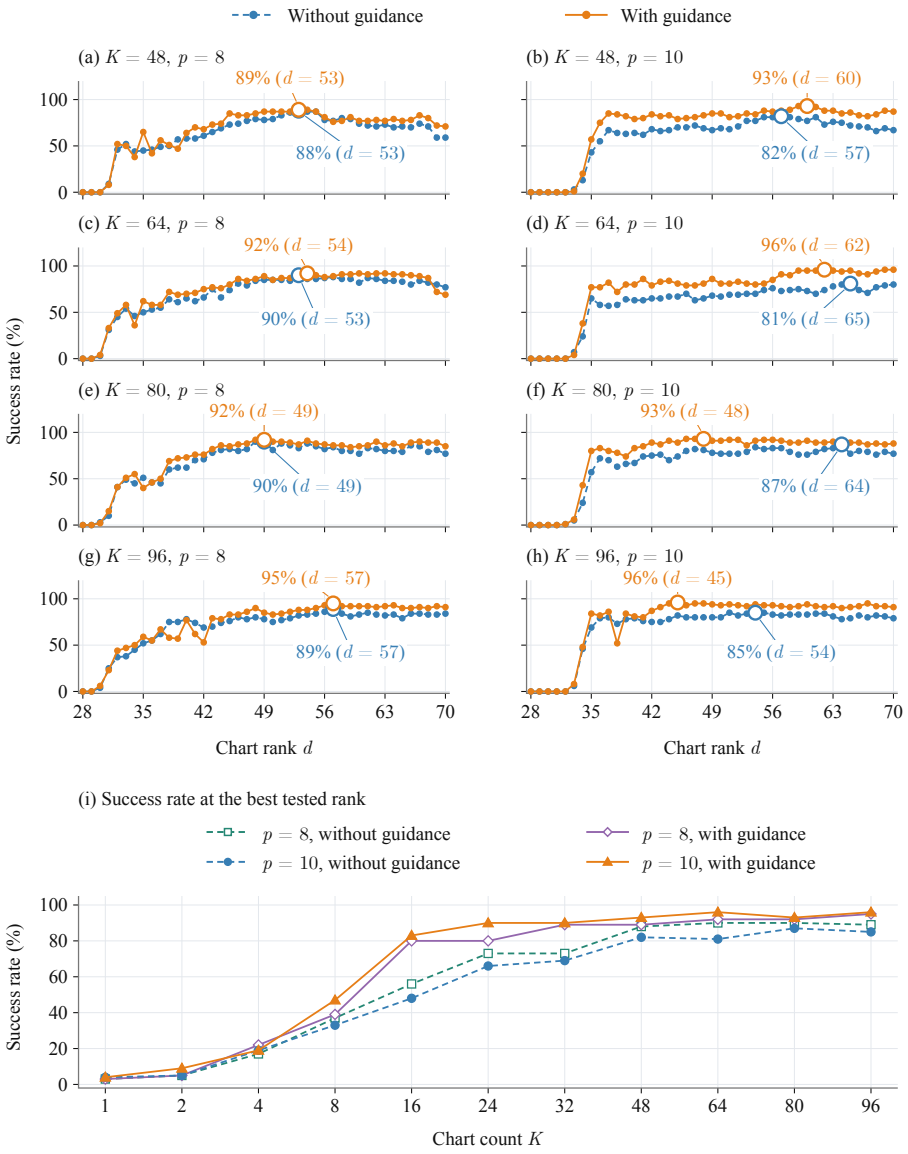


Figure 4: Success rate over 100 development seeds. (a) – (h): Success rate versus chart rank d for different chart counts K and output dimensions p, with and without graph guidance. Highlighted markers indicate the ranks selected by the highest success rate. (i): Success rate versus chart count K for all four configurations, using the best rank at each K.

Appendix B. Additional Reacher Experiments

B.1. Chart counts and ranks

The chart count K determines how finely the offline trajectories are partitioned, while the chart rank determines the dimension of each local subspace. Figure 4 reports success rates on 100 development simulations. Panels (a) – (h) show success rates against chart rank for different chart counts and output dimensions, with and without graph guidance. Each panel shows success rates for chart ranks in $[28, 70]$. Lower ranks yield poor performance, whereas sufficiently large ranks produce

a broad region of higher success rates. Increasing the rank further does not consistently improve performance. These development simulations inform the chart numbers and ranks used in Table 1. The unguided $p = 8$ configuration uses $K = 64$ and rank $d = 53$, as introduced in Section 4. The other selected (K, d) pairs are $(96, 57)$ for guided $p = 8$, $(80, 64)$ for unguided $p = 10$, and $(64, 62)$ for guided $p = 10$. Table 1 evaluates these four configurations on 500 test simulations, separately from the development simulations used here for parameter selection.

B.2. Effects of Reference Planning

We isolate the effect of graph-guided reference planning by fixing $p = 8, K = 64, d = 53$, and evaluating both configurations on 500 test simulations. Table 3 shows that guidance increases the success rate from 90.0% to 96.2%, recovering 36 failed simulations while causing 5 previously successful simulations to fail. It also reduces upper-percentile errors. Figure 5 illustrates the effect of graph guidance on representative simulations.

Graph guidance	Success		Final-30 mean error (mm)			
		Rate	Median	90th percentile	95th percentile	99th percentile
No	450	90.0%	**1.81**	10.05	218.27	299.30
Yes	**481**	**96.2%**	2.04	**4.49**	**7.40**	**38.00**

Table 3: Effect of graph guidance on 500 test simulations (seeds 10001–10500), with $p = 8$, $K = 64$, $d = 53$. Both configurations use the same atlas, controller parameters, initial states, and targets. Error statistics summarize final-30 mean fingertip errors across all simulations.

B.3. Implementation details of algorithm comparison

B.3.1. SELECT-DPC AND CONTEXTUAL-DPC

Select-DPC (Näf et al., 2025) alternates between data selection and predictive optimization at each control step. Our implementation uses the authors' publicly available code[2], adapted to our Reacher benchmark. In each iteration, it selects n_s columns of the offline mosaic Hankel matrix H based on the latest predicted input-output trajectory. Denote the selected data matrix by H_{sel}. The optimization problem of Select-DPC is

$$\min_{g,u^{\text{f}},y^{\text{f}},\sigma} \quad (y^{\text{f}} - y^{\text{ref}})^\top Q(y^{\text{f}} - y^{\text{ref}}) + (u^{\text{f}})^\top R u^{\text{f}} + \lambda_{\text{ini}}\|\sigma\|_1 + \lambda_1\|g\|_1 + \lambda_\Pi\|(I - \Pi)g\|_2^2$$

$$\text{s.t.} \quad H_{\text{sel}} g = \begin{bmatrix} u_{\text{ini}} \\ y_{\text{ini}} + \sigma \\ u^{\text{f}} \\ y^{\text{f}} \end{bmatrix}, \quad \mathbf{1}^\top g = 1, \quad u^{\text{f}} \in \mathcal{U}, \quad y^{\text{f}} \in \mathcal{Y}.$$

The projection regularizer (Markovsky et al., 2023) is defined by

$$\Pi = M^\dagger M, \qquad M = \begin{bmatrix} H_{\text{sel}}(\mathcal{J}, :) \\ \mathbf{1}^\top \end{bmatrix}, \qquad \mathcal{J} = \{1, \ldots, (m+p)T_{\text{ini}} + mT_{\text{f}}\}.$$

2. https://github.com/naefjo/choose-wisely-paper

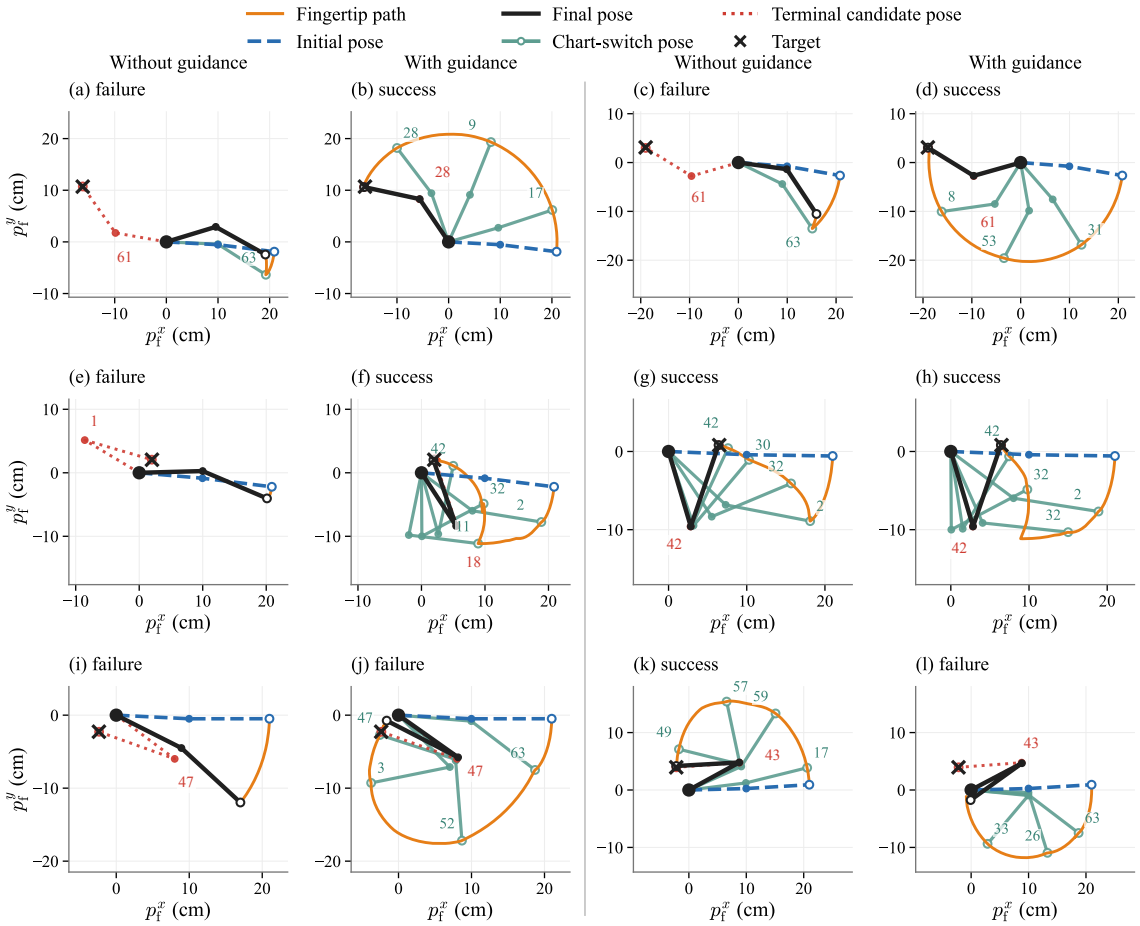


Figure 5: Effect of graph guidance on selected Reacher test simulations with $p = 8, K = 64, d = 53$. The first and third columns show simulations without guidance, whereas the second and fourth columns show simulations with guidance. Numbers labeling the chart-switch and terminal reference poses denote their chart IDs. These examples illustrate recovery, unchanged success or failure, and regression.

Contextual-DPC (Beerwerth and Alrifaee, 2026) selects n_s offline trajectories by measuring how closely their past outputs match the recent measurements and their future outputs match the reference. The two distances are normalized and combined to rank the candidate trajectories, see (Beerwerth and Alrifaee, 2026, Section II-C and Algorithm 1) for details. Unlike Select-DPC, data selection and predictive optimization are performed only once per control step. The optimization problem of Contextual-DPC is

$$\min_{g,u^{\mathrm{f}},y^{\mathrm{f}},\sigma} \quad (y^{\mathrm{f}} - y^{\mathrm{ref}})^\top Q (y^{\mathrm{f}} - y^{\mathrm{ref}}) + (u^{\mathrm{f}})^\top R u^{\mathrm{f}} + (\Delta u^{\mathrm{f}})^\top R\, \Delta u^{\mathrm{f}} + \lambda_g \|g\|_2 + \lambda_{\mathrm{ini}} \|\sigma\|_1$$

$$\text{s.t.} \quad H_{\mathrm{sel}} g = \begin{bmatrix} u_{\mathrm{ini}} \\ y_{\mathrm{ini}} + \sigma \\ u^{\mathrm{f}} \\ y^{\mathrm{f}} \end{bmatrix}, \qquad \mathbf{1}^\top g = 1, \quad u^{\mathrm{f}} \in \mathcal{U}, \quad y^{\mathrm{f}} \in \mathcal{Y},$$

where $\Delta u^{\mathrm{f}} := \big(u_t - u_{t-1}, u_{t+1} - u_t, \dots, u_{t+T_{\mathrm{f}}-1} - u_{t+T_{\mathrm{f}}-2}\big)$.

B.3.2. IMPLEMENTATION

We evaluate both baselines on 500 test simulations. Implementation details are given below, and Table 2 reports tracking performance and online computation times.

Select-DPC. Since Näf et al. (2025) include a Reacher benchmark, we use their publicly released implementation and dataset. We use the hyperparameters reported in (Näf et al., 2025, Appendix D):

$$Q = I_{T_{\mathrm{f}}} \otimes \mathrm{diag}(0,0,0,0,10,10,40000,40000), \qquad R = 10 I_{mT_{\mathrm{f}}},$$
$$\lambda_1 = 10, \quad \lambda_\Pi = 10^4, \quad \lambda_{\mathrm{ini}} = 5 \times 10^6, \quad T_{\mathrm{ini}} = 2, \quad T_{\mathrm{f}} = 15, \quad n_s = 200, \quad n_{\max} = 5,$$
$$\mathcal{U} = [-0.5, 0.5]^{mT_{\mathrm{f}}}, \qquad \mathcal{Y} = \left\{ y^{\mathrm{f}} \in \mathbb{R}^{8T_{\mathrm{f}}} \;\middle|\; y^{\mathrm{f}}(8j+8) \le 0.1, \quad j = 0, \dots, T_{\mathrm{f}} - 1 \right\}.$$

Their dataset contains 200 episodes of 200 steps collected using i.i.d. inputs uniformly distributed over $[-0.1, 0.1]^2$. This dataset differs from ours, which uses exploration inputs in $[-1, 1]^2$. We tested Select-DPC with our offline dataset, but did not achieve successful tracking. This is likely due to the hyperparameters in Select-DPC being data-dependent. We therefore used the authors' original dataset for comparison. Testing Atlas-DeePC on the original dataset of Näf et al. (2025) yielded similar results to those reported in Table 2. The trajectories are assembled into a mosaic-Hankel matrix with $L = T_{\mathrm{ini}} + T_{\mathrm{f}} = 17$.

Contextual-DPC. We adapt Beerwerth and Alrifaee (2026) to Reacher using our offline dataset. Following their formulation, we stack the output and input as $(y_t, u_t) \in \mathbb{R}^{m+p}$ and apply the data normalization described in their Section II-D. We use (Beerwerth and Alrifaee, 2026, Algorithm 1) for data selection. Following the output construction in (Beerwerth and Alrifaee, 2026, Section II-D, "Incremental Input Representation"), the tracking reference uses the same $y^{\mathrm{ref}} \in \mathbb{R}^8$ constructed in Section 4. The hyperparameters are tuned on 100 development simulations and set to

$$Q = I_{T_{\mathrm{f}}} \otimes \mathrm{diag}(0.01, 0.01, 0.01, 0.01, 10^{-4}, 10^{-4}, 100, 100, 0, 0), \quad R = 0.1 I_{mT_{\mathrm{f}}},$$
$$\lambda_g = 20, \quad \lambda_{\mathrm{ini}} = 100, \quad T_{\mathrm{ini}} = 2, \quad T_{\mathrm{f}} = 10, \quad n_s = 200.$$

For tracking the target, contextual-DPC uses the same fixed terminal reference as unguided Atlas-DeePC constructed in Section 4.

Appendix C. F1TENTH Raceline Tracking

This experiment evaluates Atlas-DeePC on the raceline-tracking task which is also considered by contextual-DPC (Beerwerth and Alrifaee, 2026).

Task. We consider raceline tracking in F1TENTH Gym (O'Kelly et al., 2020) using nonlinear single-track dynamics (Limebeer and Sharp, 2006) with sampling period $\Delta t = 0.1\,\mathrm{s}$. The input $u_t = (a_t, \delta_t)$ consists of acceleration a_t ($\mathrm{m/s^2}$) and steering angle δ_t (rad). The output $y_t = (p_t^x, p_t^y, v_t, \psi_t) \in \mathbb{R}^4$ contains the vehicle position $p_t = (p_t^x, p_t^y)$, speed, and heading, as illustrated in Figure 6. We express positions and headings in a local coordinate frame. Let τ denote the time

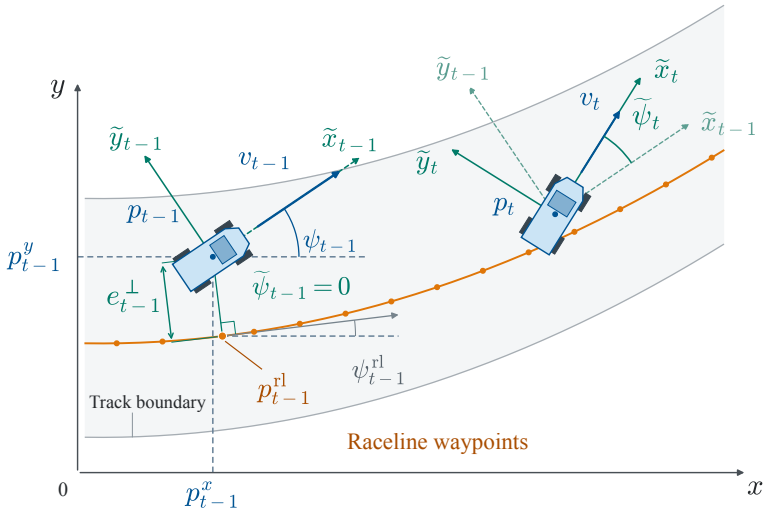


Figure 6: Vehicle motion and local coordinate frames. Each local frame is centered at the vehicle position and aligned with its heading. At time t, the light-green dashed arrows show the directions of the previous frame, translated for comparison. Relative to that frame, $\widetilde{\psi}_{t-1} = 0$ and $\widetilde{\psi}_t = \psi_t - \psi_{t-1}$. Orange dots denote raceline waypoints. The cross-track error $e^{\perp}_{t-1}$ is measured perpendicular to the raceline heading ψ^{rl}_{t-1} at the selected waypoint p^{rl}_{t-1}.

index of a sample and a the time index defining the frame, whose origin is p_a and whose positive $\widetilde{x}$-axis points along ψ_a. The transformed position and heading are

$$\widetilde{p}_\tau = \begin{bmatrix} \cos\psi_a & \sin\psi_a \\ -\sin\psi_a & \cos\psi_a \end{bmatrix} (p_\tau - p_a), \qquad \widetilde{\psi}_\tau = \psi_\tau - \psi_a.$$

The components of $\widetilde{p}_\tau$ give the position along the local axes, and $\widetilde{\psi}_\tau$ is the heading relative to the positive $\widetilde{x}$-axis. For each offline trajectory h_i, we transform all its position and heading components using the position and heading at the T_{ini}-th time step to define the local frame. Online, we set $a = t-1$, using the latest measured position p_{t-1} and heading ψ_{t-1} to define the frame illustrated by the $\widetilde{x}_{t-1}$- and $\widetilde{y}_{t-1}$-axes in Figure 6. The reference output sequence is constructed from the raceline waypoints and prescribed speeds provided with F1TENTH Gym. Both the recent measurements and the reference output sequence are transformed using this same frame. To measure tracking error, we select a closest raceline waypoint, whose position p^{rl}_t and heading ψ^{rl}_t are illustrated in Figure 6. The signed lateral error and its root-mean-square error (RMSE) over a simulation of length T_{sim} are

$$e^{\perp}_t := \begin{bmatrix} -\sin\psi^{\mathrm{rl}}_t & \cos\psi^{\mathrm{rl}}_t \end{bmatrix} (p_t - p^{\mathrm{rl}}_t), \qquad \mathrm{RMSE} := \sqrt{\frac{1}{T_{\mathrm{sim}}} \sum_{t=0}^{T_{\mathrm{sim}}-1} (e^{\perp}_t)^2}.$$

The vector $(-\sin\psi^{\mathrm{rl}}_t, \cos\psi^{\mathrm{rl}}_t)$ has unit length and points perpendicular to the raceline heading. Thus, $e^{\perp}_t$ measures the vehicle's signed lateral displacement from the selected waypoint, and RMSE summarizes its magnitude over the simulation. A simulation lap is *successful* if the vehicle completes one lap with $|e^{\perp}_t| \le 1$ m for all t.

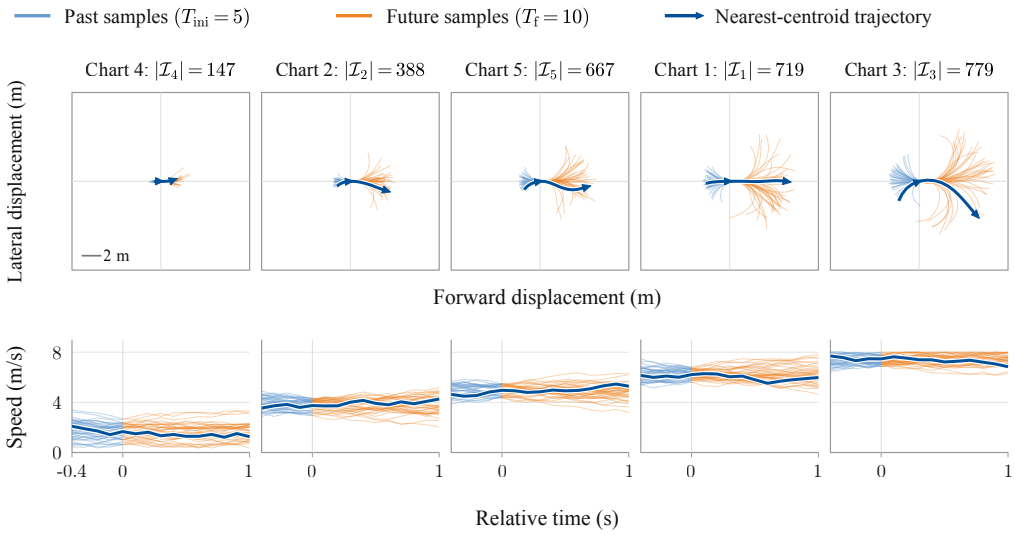


Figure 7: Behavioral charts of the noise-free F1TENTH data with $K = 5$. Vehicle paths in local coordinates (top) and speed profiles (bottom). Thin curves show 40 sampled trajectories per chart, with initialization samples ($T_{\text{ini}} = 5$) in blue and future samples ($T_{\text{f}} = 10$) in orange. Bold dark-blue curves show the nearest-centroid trajectory $h_{i_k^*}$, where $i_k^* \in \operatorname{argmin}_{i \in \mathcal{I}_k} \|z_i - c_k\|_2$. Arrows indicate travel direction, and $|\mathcal{I}_k|$ denotes the number of trajectories in chart $\mathcal{S}_k$.

Behavioral atlas. We collect trajectories from $N = 20$ independent episodes, each of length $N_j = 150$. The initial speed is sampled uniformly from $[4, 8]\,\text{m/s}$. Acceleration inputs are sampled independently and uniformly from $[-3, 3]\,\text{m/s}^2$. The steering inputs are generated by

$$\delta_t = \min\{0.3, \max\{-0.3, \eta_t\}\}, \text{ where } \eta_t = 0.8\eta_{t-1} + 0.6\xi_t \text{ and } \xi_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 0.12^2).$$

Here, $\eta_{-1} = 0$, and $\xi_t \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 0.12^2)$ means that ξ_t are independent and identically distributed Gaussian random variables with zero mean and standard deviation 0.12. At the sampling period of $0.1\,\text{s}$, independently sampled steering commands can cause abrupt steering changes and rapid changes in vehicle heading. We therefore generate temporally correlated steering inputs to obtain smoother steering variations. To assess robustness to sensor measurement noise, we add noise to the position, speed, and heading measurements before coordinate alignment, yielding

$$y_t = \big(p_t^x + \sigma s_1 \epsilon_{t,1},\ p_t^y + \sigma s_2 \epsilon_{t,2},\ v_t + \sigma s_3 \epsilon_{t,3},\ \psi_t + \sigma s_4 \epsilon_{t,4}\big), \qquad \epsilon_{t,j} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1).$$

Here, $\sigma \geq 0$ denote the noise level. The quantities s_1, s_2, s_3, s_4 are the standard deviations of $p_t^x, p_t^y, v_t, \psi_t$, respectively, computed across all time steps in the noise-free offline data before coordinate alignment. We set $T_{\text{ini}} = 5$, $T_{\text{f}} = 10$. For atlas construction, the feature vector z_i contains only the speed at the T_{ini}-th sample of h_i, i.e.,

$$\mathcal{R} := \{(m + p)(T_{\text{ini}} - 1) + m + 3\}, \qquad Z = H(\mathcal{R}, :) \in \mathbb{R}^{1 \times T}.$$

For each value of σ, we construct Z from the noisy offline data and cluster it using mini-batch K-means, and construct a rank-d behavioral chart $\mathcal{S}_k$ (Figure 7) from each local data matrix $H^{\mathcal{I}_k}$ following the procedure in Section 3.

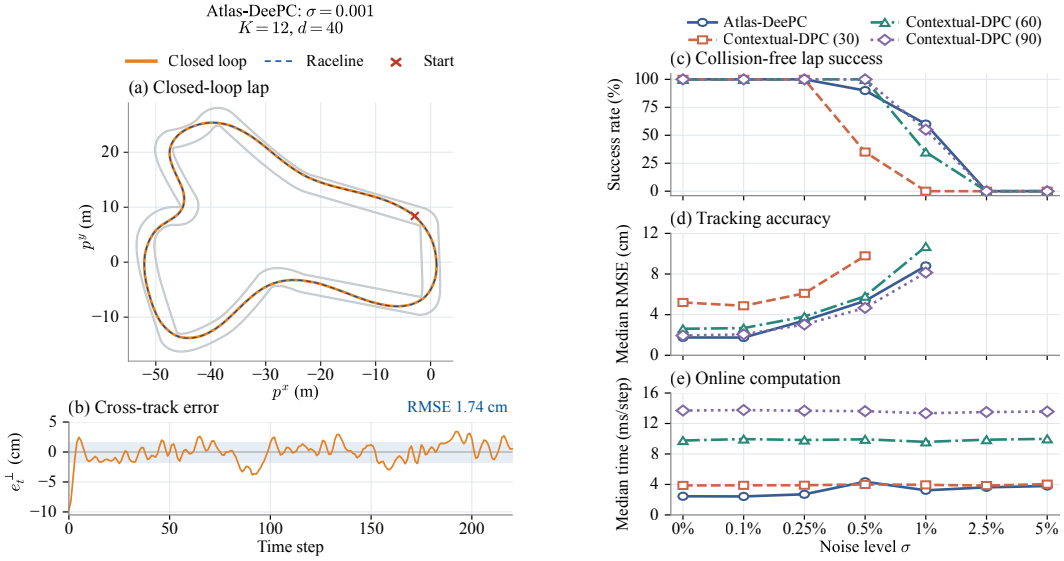


Figure 8: F1TENTH tracking and noise sensitivity. (a)–(b) Atlas-DeePC simulation at $\sigma = 0.001$, with $K = 12$, $d = 40$, and seed 542: closed-loop path and signed cross-track error. The shaded band indicates $\pm$ the simulation RMSE of 1.74 cm. (c)–(e) Success rate, median cross-track RMSE over successful simulations, and median computation time over 20 test simulations for each combination of noise level and controller configuration. Missing RMSE points indicate no successful simulations. Numbers in parentheses following Contextual-DPC indicate the number of data columns.

Controller. The controller tracks the prescribed raceline and speed profile, expressed in the same local coordinate frame as the measurements. The chart $\mathcal{S}_{k_t}$ is selected by minimizing the projection residual (5) with $L_{\mathrm{sel}} = T_{\mathrm{ini}}$. The cost weights are

$$Q = I_{T_{\mathrm{f}}} \otimes \mathrm{diag}(100, 100, 1, 1), \qquad R = I_{T_{\mathrm{f}}} \otimes \mathrm{diag}(0.1, 10), \quad \lambda_{\mathrm{ini}} = 100.$$

The constraints are $\mathcal{U} = \big([-3, 3] \times [-0.4189, 0.4189]\big)^{T_{\mathrm{f}}}$, and $\mathcal{Y} = \mathbb{R}^{4T_{\mathrm{f}}}$. We use the control policy $\pi_{\mathcal{S}_{k_t}}$ (6), with the quadratic program solved using Clarabel.

Results. We compare Atlas-DeePC with Contextual-DPC at noise levels $\sigma \in \{0, 0.001, 0.0025, 0.005, 0.01, 0.025, 0.05\}$. Contextual-DPC uses 30, 60, or 90 selected trajectory columns. At each noise level, both methods use the same noisy offline dataset and are evaluated on 20 test simulations. The Atlas-DeePC parameters (K, d) are selected separately at each noise level using development simulations. No reference planning is used. Figure 8 (a)–(b) shows an Atlas-DeePC simulation at $\sigma = 0.001$, with $K = 12$, $d = 40$. The vehicle completes a collision-free lap with a cross-track RMSE of 1.74 cm. Panels (c)–(e) summarize success rates, median cross-track RMSE over successful simulations, and median online computation times. Atlas-DeePC achieves tracking errors comparable to Contextual-DPC using 90 columns, while requiring computation times generally lower than those of the 30-column configuration. In these experiments, Atlas-DeePC selects the same chart throughout each simulation, i.e., chart 2 for $\sigma = 0$ and chart 1 for all $\sigma > 0$ (Table 4). For all tested nonzero noise levels, the selected subspaces $\mathcal{S}_k^{\mathrm{sel}}$ are identical to numerical precision, therefore they yield the same projection residual in (5) for any trajectory $w_{[t-L_{\mathrm{sel}}, t-1]}$, and the smallest-index rule selects chart 1. This example demonstrates competitive tracking performance with low online computation, while showing that both methods remain sensitive to increasing noise.

σ	K	d	Maximum pairwise distance	Selected chart
0	2	42	1.4142	2
0.001	12	40	6.63×10^{-15}	1
0.0025	13	42	6.13×10^{-15}	1
0.005	14	60	7.16×10^{-15}	1
0.01	10	44	6.24×10^{-15}	1
0.025	3	68	4.73×10^{-15}	1
0.05	8	72	5.38×10^{-15}	1

Table 4: F1TENTH chart-selection subspaces with $L_{\mathrm{sel}} = T_{\mathrm{ini}} = 5$ and $T_{\mathrm{f}} = 10$. Maximum pairwise distance denotes $\max_{1 \le k < k' \le K} d_\kappa(\mathcal{S}_k^{\mathrm{sel}}, \mathcal{S}_{k'}^{\mathrm{sel}})$, computed using the chordal distance d_κ defined in Section 5. For each noise level, the column "selected chart" reports the chart index which remains constant over all time steps according to the chart selection rule (5). Chart 1 is selected at every time step in all 20 test simulations across all tested nonzero noise levels, whereas chart 2 is selected throughout all noise-free simulations.